

**Case Study
On**

Restaurant Management System

INTRODUCTION

- The Restaurant Management System is a web-based application developed using Spring Boot, MySQL, HTML, CSS, and JavaScript. The project aims to simplify restaurant operations such as menu management and order handling through a user-friendly interface.
- It allows administrators to add, view, and delete menu items, place customer orders, and track order status efficiently. The backend is built using RESTful APIs with Spring Data JPA for database interaction, while the frontend provides a responsive and interactive experience.
- This system improves accuracy, reduces manual effort, and demonstrates full-stack development with a clear separation of frontend, backend, and database layers.

ABSTRAT

The Restaurant Management System is a full-stack web application designed to automate and streamline restaurant operations. Developed using Spring Boot, MySQL, HTML, CSS, and JavaScript, the system enables efficient management of menu items and customer orders. It supports features such as adding and deleting menu items, placing orders, and tracking order status in real time. The backend uses RESTful APIs and Spring Data JPA for secure and efficient database interaction, while the frontend provides a simple and responsive user interface. This project reduces manual work, improves accuracy, and demonstrates practical implementation of modern web application development concepts.

What we are going to build: Client Requirement

- The client requires a web-based Restaurant Management System to efficiently manage daily restaurant operations. The system should allow the administrator to add, view, and delete menu items along with their categories and prices.
- It must support placing customer orders and tracking their status from order placement to completion. The application should provide a user-friendly interface for easy navigation between pages such as Home, Menu, and Orders.
- Data must be stored securely in a MySQL database, and the system should ensure fast and accurate processing. The solution should reduce manual effort, improve order accuracy, and provide real-time updates for restaurant staff.

What we are going to build: Some technical terms.

The project involves developing a full-stack **web-based Restaurant Management System** using **Spring Boot** for backend processing and **HTML, CSS, and JavaScript** for the frontend interface. The backend will follow a **layered architecture** consisting of Controller, Service, Repository, and Entity layers. **Spring Data JPA** will be used for ORM (Object Relational Mapping) to interact with a **MySQL relational database**.

The application will expose **RESTful APIs** for menu and order management, enabling CRUD operations. **Hibernate** will handle database transactions, while **Apache Tomcat** will act as the embedded web server. The system will support real-time data persistence, request handling, and page navigation through HTTP endpoints.

What are the technologies and tools we are going use ?

- Framework: **Spring Boot** Java Framework.
 - Java 8+
 - Maven
 - STS
 - Apache Tomcat
 - Spring core, Spring security(jwt) , Spring data JPA(Hibernate) et.c
- MySQL Database.
- Postman Rest Client
- Swagger

SYSTEM REQUIREMENTS

SOFTWARE REQUIREMENT :

- ❑ **Back End:** Spring Boot , JPA Hibernate and Postman Tool
- ❑ **DB Server:** MySQL Server 8.0.CE
- ❑ **Web Server:** Apache Tomcat
- ❑ **IDE:** Spring Tool Suite(STS)
- ❑ **Browser:** Google Chrome
- ❑ **Test:** JUnit

PROJECT MODULE

The Restaurant Management System is divided into multiple functional modules to ensure clear organization, easy maintenance, and smooth operation of the application.

1. Home Module

The Home module acts as the landing page of the application. It provides an overview of the system and allows users to navigate easily to other modules such as Menu Management and Order Management through a common navigation bar.

2. Menu Management Module

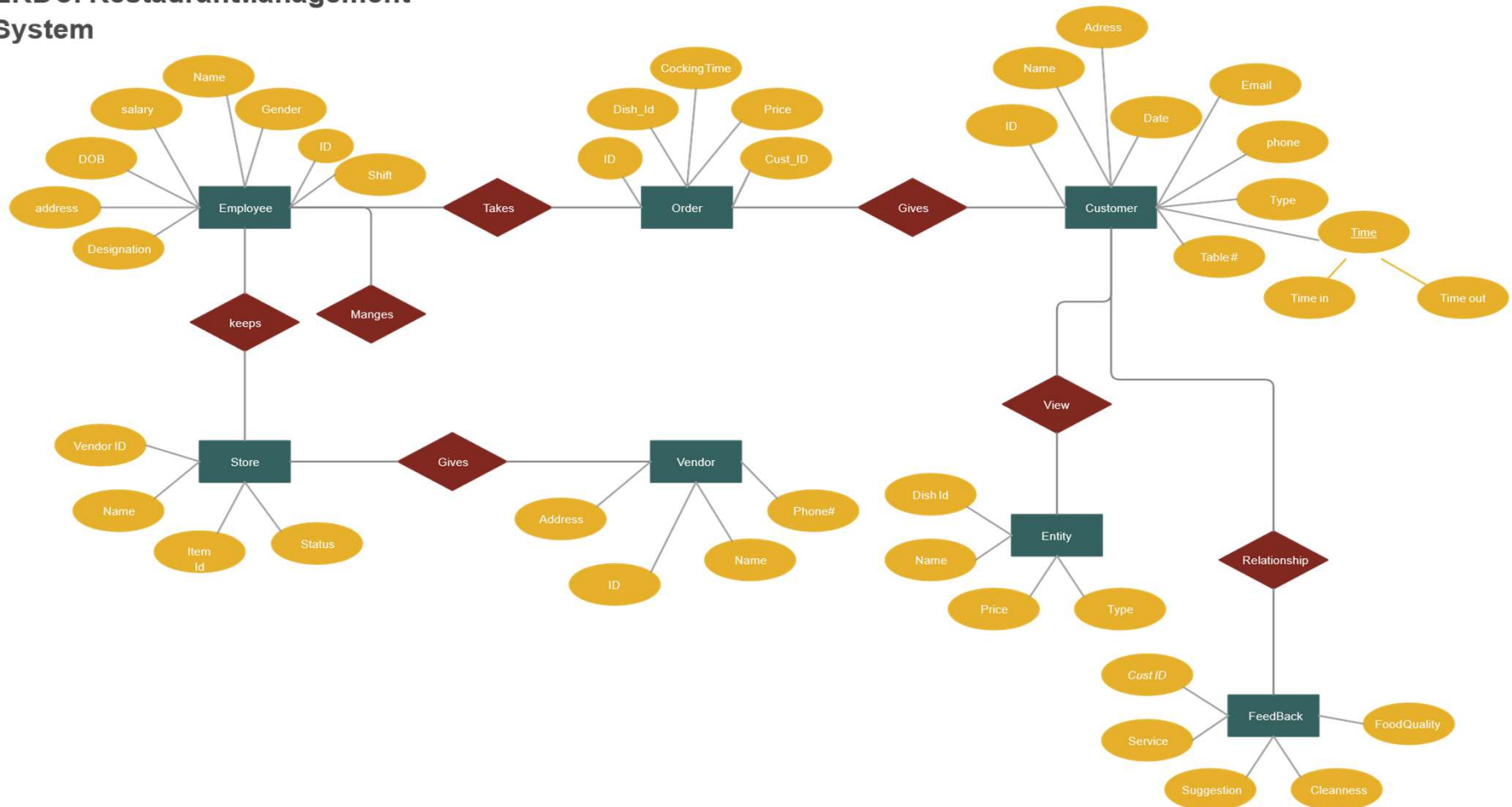
This module allows the administrator to manage restaurant menu items. It supports adding new food items with details such as name, category, and price. The module also enables viewing and deleting menu items. All operations are performed using REST APIs connected to the MySQL database.

3. Order Management Module

The Order Management module handles customer orders. It allows placing new orders, calculating the total bill amount, and tracking the order status such as placed, preparing, and served. This module ensures efficient order processing and real-time updates.

ER DIAGRAM

ERD of Restaurant Management System



ADMIN MODULE

The Admin Module is responsible for managing and controlling the core operations of the Restaurant Management System. It allows the administrator to maintain the restaurant's menu and monitor order activities. Through this module, the admin can add new menu items by specifying details such as item name, category, and price. The admin can also view and delete existing menu items when required.

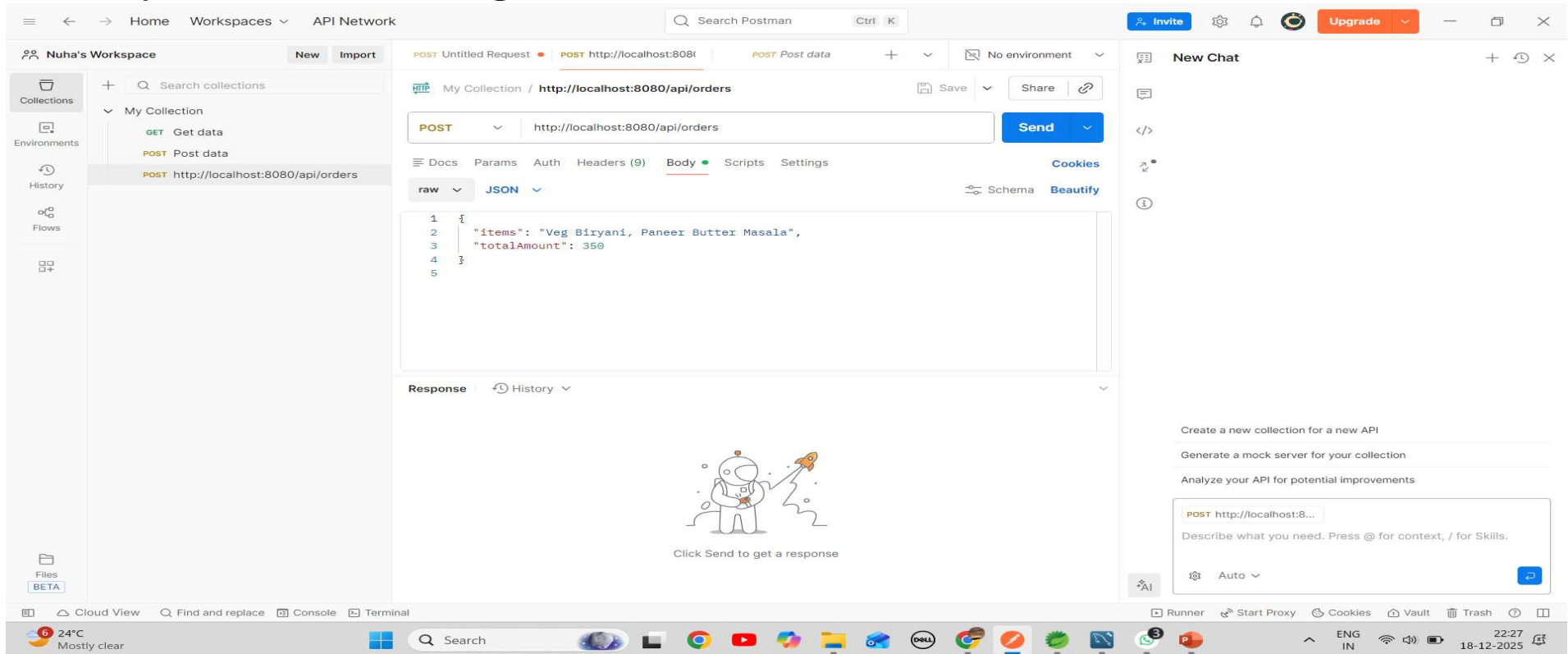
In addition, the admin can view all customer orders and track their status in real time, such as placed, preparing, and served. This module ensures efficient control over restaurant operations, improves data accuracy, and reduces manual effort. It acts as the central authority for managing data and maintaining smooth system functionality.

Menu Management Module

- The Menu Management Module is a core component of the Restaurant Management System that enables efficient handling of food items offered by the restaurant. This module allows the administrator to add new menu items by specifying details such as item name, category, and price.
- It also provides functionality to view all available menu items in a structured format and delete items that are no longer required. The module interacts with the backend through RESTful APIs and stores data securely in a MySQL database. This ensures accurate menu updates, reduces manual effort, and helps maintain consistency in restaurant operations.

Http Request Methods

HTTP defines a set of **request methods** to indicate the desired action to be performed for a given resource



COMPANY MODULE

- After completing the **Menu Management Module**, we move to the **Order Management Module**. In the Order Management Module, order-related data is handled through REST APIs using tools like **Postman** for testing and validation. Orders are created using **POST mapping**, retrieved using **GET mapping**, updated using **PUT mapping** (for order status updates), and removed using **DELETE mapping** if required.
- All order-related operations interact with the MySQL database through Spring Data JPA. **JUnit testing** can be used to test the methods implemented in the Order Management Module to ensure correctness and reliability. After completing the Order Module, additional features such as order status tracking and billing can be implemented to enhance system functionality.

TRAINING MODULE

- After completing Company Module , we can move into Training Module.
- In Training Module we can add data by using Postman tool .
- In Training Module can add the training details by using Post Mapping , we can using Get Mapping by Collect list of training Details , we can update the training details by using Put Mapping and we can delete training details by using Delete mapping.
- We can use Junit test for all methods in Training Module.
- After Completing Training Module we go with Restaurant Module .
Module is a child table of Training Module .

DATABASE MODULE

- The Database Module is responsible for storing and managing all persistent data related to menu items and customer orders.
- The system uses a MySQL relational database, integrated with Spring Data JPA and Hibernate for efficient data handling.
- This module ensures data consistency, reliability, and secure storage while supporting CRUD operations required by the application.

DATA DICTIONARY

TABLES OF DATABASE

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Administration Schemas

Information

No object selected

Object Info Session

Query Editor

```
1 CREATE DATABASE restaurant_db;
2 USE restaurant_db;
3 SHOW DATABASES;
4 SHOW TABLES;
5 DELETE FROM menu_item;
6 SELECT * FROM menu_item;
7 SELECT * FROM orders;
8 UPDATE orders
9 SET price = 0
10 WHERE price IS NULL;
11
12 ALTER TABLE orders
13 MODIFY price DOUBLE NOT NULL,
14 MODIFY quantity INT NOT NULL;
```

Result Grid

Tables_in_restaurant_db

- menu_item
- orders

Result 1

Output

#	Time	Action	Message	Duration / Fetch
4	21:03:11	UPDATE orders SET quantity = 1 WHERE quantity IS NULL	4 row(s) affected Rows matched: 4 Changed: 4 Warnings: 0	0.015 sec
5	21:03:31	ALTER TABLE orders MODIFY price DOUBLE NOT NULL, MODIFY quantity INT NOT NULL	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.281 sec
6	22:31:00	SHOW TABLES	2 row(s) returned	0.250 sec / 0.000 sec

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

24°C Mostly clear

Search

ENG IN

22:31 18-12-2025

ORDERS DATABASE

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Administration Schemas

Information

No object selected

SQL Editor

```
1 • CREATE DATABASE restaurant_db;
2 • USE restaurant_db;
3 • SHOW DATABASES;
4 • SHOW TABLES;
5 • DELETE FROM menu_item;
6 • SELECT * FROM menu_item;
7 • SELECT * FROM orders;
8 • UPDATE orders
9   SET price = 0
10  WHERE price IS NULL;
11
12 • ALTER TABLE orders
13   MODIFY price DOUBLE NOT NULL,
14   MODIFY quantity INT NOT NULL;
```

Result Grid

#	id	items	order_time	total_amount	status	customer_name	item_name	price	quantity
1	1	Veg Biryani, Paneer Butter Masala	2025-12-14 22:47:47.065000	350	NULL	NULL	NULL	0	1
2	2	Veg Biryani, Paneer Butter Masala	2025-12-15 00:13:56.092337	350	NULL	NULL	NULL	0	1
3	3	Veg Biryani, Paneer Butter Masala	2025-12-16 09:56:48.058407	350	NULL	NULL	NULL	0	1
4	4	Veg Biryani, Paneer Butter Masala	2025-12-16 11:40:35.607069	350	NULL	NULL	NULL	0	1

orders 2

Output

Action Output

#	Time	Action	Message	Duration / Fetch
5	21:03:31	ALTER TABLE orders MODIFY price DOUBLE NOT NULL, MODIFY quantity INT NOT NULL	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.281 sec
6	22:31:00	SHOW TABLES	2 row(s) returned	0.250 sec / 0.000 sec
7	22:31:44	SELECT * FROM orders LIMIT 0, 1000	4 row(s) returned	0.015 sec / 0.000 sec

Object Info Session

24°C Mostly clear

Search

22:31 18-12-2025

GET METHOD FOR ADD DATA

The screenshot displays the Postman API client interface. At the top, the navigation bar includes 'Home', 'Workspaces', and 'API Network'. A search bar labeled 'Search Postman' with a 'Ctrl K' shortcut is present. The main workspace shows a GET request to 'http://localhost:8080/api/orders'. The 'Body' tab is selected, showing a raw JSON body:

```
{  "items": "Veg Biryani, Paneer Butter Masala",  "totalAmount": 350}
```

. The 'Send' button is visible. Below the request, the response is shown with a status of '200 OK', a response time of '477 ms', and a size of '365 B'. The response body is a JSON array:

```
[  {    "id": 1,    "items": "Veg Biryani, Paneer Butter Masala",    "orderTime": "2025-12-14T22:47:47.065",    "totalAmount": 350.0  }]
```

. The bottom status bar shows system information: '23°C Partly cloudy', a search bar, and various application icons. The system clock indicates '00:13 15-12-2025'.

POST METHOD FOR GET ALL DATA

The screenshot displays the Postman application interface. At the top, the navigation bar includes 'Home', 'Workspaces', and 'API Network'. A search bar and a 'Search Postman' button are also present. The main workspace shows a POST request to 'http://localhost:8080/api/orders'. The request body is a JSON object:

```
{  "items": "Veg Biryani, Paneer Butter Masala",  "totalAmount": 350}
```

. The response is a 200 OK status with a response time of 7.78 s and a body size of 367 B. The response body is a JSON object:

```
{  "id": 2,  "items": "Veg Biryani, Paneer Butter Masala",  "orderTime": "2025-12-15T00:13:56.0923366",  "totalAmount": 350.0}
```

. The bottom status bar shows the system clock as 00:14 on 15-12-2025, along with weather information (23°C, Partly cloudy) and various system icons.

Home Workspaces API Network

Search Postman Ctrl K

POST http://localhost:8080/c POST http://localhost:8080/s

http://localhost:8080/api/orders

POST http://localhost:8080/api/orders

Docs Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "items": "Veg Biryani, Paneer Butter Masala",
3   "totalAmount": 350
4 }
5
```

Body Cookies Headers (8) Test Results

200 OK 7.78 s 367 B

{ } JSON Preview Visualize

```
1 {
2   "id": 2,
3   "items": "Veg Biryani, Paneer Butter Masala",
4   "orderTime": "2025-12-15T00:13:56.0923366",
5   "totalAmount": 350.0
6 }
```

Outlook (new)

Runner Start Proxy Cookies Vault Trash

23°C Partly cloudy Search

ENG IN 00:14 15-12-2025

JUNIT TEST CASES FOR ODERS MODULE

workspace-spring-tools-for-eclipse-4.32.2.RELEASE - restaurant-management/src/main/java/com/restaurant/service/OrderServiceImpl.java - Spring Tools for Eclipse

File Edit Source Refactor Navigate Search Project Run Window Help

Project Explorer

- curd [boot]
- restaurant-management [boot]
 - src/main/java
 - com.restaurant
 - RestaurantManagementApp
 - com.restaurant.controller
 - MenuItemController.java
 - OrderController.java
 - com.restaurant.entity
 - MenuItem.java
 - Order.java
 - com.restaurant.repository
 - MenuItemRepository.java
 - OrderRepository.java
 - com.restaurant.service
 - MenuItemService.java
 - MenuItemServiceImpl.java
 - OrderService.java
 - OrderServiceImpl.java
 - src/main/resources
 - static
 - images
 - restaurant-bg.jpg
 - index.html
 - menu.html
 - orders.html
 - style.css
 - templates
 - application.properties
 - src/test/java
 - JRE System Library [JavaSE-17]
 - Maven Dependencies
 - src

application... menu.html Order.java OrderReposi... OrderService... OrderService... OrderControl... orders.html index.html style.css

```
3 import java.util.List;
4
5 import org.springframework.stereotype.Service;
6
7 import com.restaurant.entity.Order;
8 import com.restaurant.repository.OrderRepository;
9
10 @Service
11 public class OrderServiceImpl implements OrderService {
12
13     private final OrderRepository repository;
14
15     public OrderServiceImpl(OrderRepository repository) {
16         this.repository = repository;
17     }
18
19     @Override
20     public Order placeOrder(Order order) {
21         order.setStatus("Placed");
22         return repository.save(order);
23     }
24
25     @Override
26     public List<Order> getAllOrders() {
27         return repository.findAll();
28     }
29
30     @Override
31     public Order updateStatus(Long id, String status) {
32         Order order = repository.findById(id)
33             .orElseThrow(() -> new RuntimeException("Order not found"));
34         order.setStatus(status);
35         return repository.save(order);
36     }
37 }
38
```

@ Javadoc Declaration Console X

restaurant-management - RestaurantManagementApplication [Spring Boot App] C:\Program Files\Java\jdk-17.0.5\bin\javaw.exe (18-Dec-2025, 10:15:32 pm elapsed: 0:22:25) [pid: 9520]

orders o1_0

Writable Smart Insert 38 : 1 : 954

24°C Mostly clear Search 22:37 18-12-2025